Who Sees What — Architecture and Usage

Prepared for Salesforce AppExchange Security Review

Package	Who Sees What — User Access, Field Security & Record Visibility Explorer
Publisher	Twinforce Solutions Limited
Contact	info@twinstack.net
Namespace	twinforce
Package Id	0HoOW00000000ZV0AY
Version submitted	0.3.0.1
Subscriber Package Version Id	04tOW00000Gs9l8YAB
Package Version Id	05iOW00000064lhYAA
API version	67.0
Metadata files	165
Code coverage	88%
Date	17 August 2026

What the product does and why

In an org with thirty permission sets and a dozen permission set groups, “why can this user see this field?” is a genuinely hard question. Answering it by hand means opening Setup pages one at a time and mentally unioning the results, and if any muting permission sets are involved a naive union gives you a confidently wrong answer.

Who Sees What does that work and shows its reasoning. An admin picks a user and drills from apps to objects to fields. Beside every permission we show which permission set, group, or profile granted it.

The tool only reads. It grants nothing, changes no permissions, and writes nothing apart from its own audit log.

How information flows

There are no external systems. Everything happens inside the subscriber’s org.

```
Admin's browser
  |  LWC: permissionExplorer + 21 child bundles
  |  Imperative Apex calls only. No fetch, no XHR.
  v
PermissionExplorerController   (@AuraEnabled entry points)
  |
  +-- PEAccessGate.assertAuthorised()   <-- ALWAYS FIRST
```

```
  |      FeatureManagement.checkPermission(
  |          'Use_Permission_Explorer')
  |      throws NOT_AUTHORISED if absent
  |
  +-- PEPermissionResolver    effective + muting parents
  +-- PEUserSearchService     user list, keyset paginated
  +-- PEAccessQueryService    object CRUD, field FLS
  +-- PEAppVisibilityService  app list (estimated)
  +-- PEUserDetailService     login history, drill-downs
  +-- PEMetadataCatalog       describe, labels, sharing
  +-- PECacheManager          cache + no-cache fallback
  +-- PEInspectionAuditLogger writes PE_Inspection_Log__c
  |
  v
Salesforce database -- all SOQL in user mode
```

Data touchpoints, in full:

Direction	What	Where it goes
Read	Permission metadata and user records	Subscriber org
Write	One audit row per inspection	PE_Inspection_Log__c, subscriber org
Cache	Resolved permissions, describe results	Subscriber's Platform Cache
Outbound	Nothing	—

The fifteen objects we query: User, UserLogin, UserRole, Profile, PermissionSet, PermissionSetAssignment, PermissionSetGroup, PermissionSetGroupComponent, PermissionSetTabSetting, ObjectPermissions, FieldPermissions, LoginHistory, AppMenuItem, TabDefinition, EntityDefinition.

Every one is permission or user metadata. We read no business data at all. To be precise about a distinction that matters: we read the ObjectPermissions and FieldPermissions rows that describe access to Account, and we never read Account records.

Authentication and encryption

The package performs no authentication. It runs inside the subscriber's existing authenticated Salesforce session. There is no login flow, no credential storage, no token handling, no Auth Provider, and no Connected App.

Encryption on data transfer does not apply, because there is no data transfer off platform. Traffic between the browser and the server is the standard Lightning transport, encrypted by the platform.

There are zero callouts. No HttpRequest, no `new Http()`, no `callout:` reference, no Named Credential, no External Credential, no Remote Site Setting. That is verifiable by searching the submitted source.

This was a deliberate choice with a real product cost, and we describe what it cost us below.

How access is controlled

Three layers, all enforced by the platform rather than by checks we wrote ourselves.

The custom permission gate. Every @AuraEnabled method begins with PEAccessGate.assertAuthorised(), which calls FeatureManagement.checkPermission('Use_Permission_Explorer') and throws a typed NOT_AUTHORISED exception if the caller lacks it.

We deliberately do not cache that decision in a static. An authorisation decision held in a mutable static outlives the context it was made in, and checkPermission is cheap enough that the guarantee is worth more than the microseconds. We had a cached version and removed it.

Sharing. All 16 implementation classes are declared `public with sharing`.

User-mode SOQL. Every query runs in user mode, so the platform enforces object and field permissions for the running admin. Inline queries use `WITH USER_MODE`; the one dynamic query uses Database.queryWithBinds with AccessLevel.USER_MODE. The practical consequence is that an admin cannot use this tool as a way around their own permissions.

What the permission set grants. PermissionCheckerPermissionSet gives:

- The custom permission Use_Permission_Explorer
- Apex class access to PermissionExplorerController and nothing else
- PE_Inspection_Log__c: Read and Create only. Edit, Delete, View All and Modify All are all withheld
- Field permissions on the three non-required audit fields
- Tab visibility for Permission_Explorer

The withheld Edit and Delete are the point of the design. A holder can read the audit log but cannot rewrite or remove their own entries.

We do not auto-assign this permission set on install, and we would rather not. An InstallHandler that silently granted the ability to inspect every user in the org would be convenient and, we think, wrong.

The audit trail

Every inspection writes one row to PE_Inspection_Log__c:

- Inspector_User__c — who ran the inspection
- Subject_User__c — who was inspected
- Action__c — RESOLVE_USER, GET_APP_OBJECTS, GET_OBJECT_DETAIL, and so on
- Inspected_At__c — UTC timestamp
- Detail__c — context, such as the object API name

Writes are buffered and flushed once per transaction.

A limitation we want to state rather than have found. If the audit insert fails, we log it with System.debug and let the inspection return. In a managed package a debug statement is not visible to the subscriber, so an audit failure is currently silent to them. We chose that over failing an admin's inspection because of a logging error, but we recognise it is a trade-off rather than a good answer, and the audit trail is the control this product's safety argument rests on. If the reviewer would prefer a surfaced warning in the response or a hard failure, we will change it.

Where the tool stops, and why

Our design rule is that we never present a guess as an answer. Where the platform cannot tell us something without a callout, the interface says so in place rather than rendering an empty result that reads as "no access."

Per-user record type visibility. This lives only in Profile and PermissionSet metadata XML. No queryable junction object exists — we verified this against the sObject list and against SetupEntityAccess, which covers Apex classes, pages, tabs, custom permissions and connected apps but not record types. Reading it needs the Metadata or Tooling API, which means a callout. So we list the record types that exist on the object beneath a standing notice that record type access is not analysed, naming recordTypeVisibilities as where the answer actually lives.

Per-permission-set app visibility. Same reason. We derive the app list from AppMenuItem and label every row "visibility estimated," with a notice above the column.

Login history. Salesforce restricts LoginHistory to users with Manage Users, and we do not grant that permission. Where the running admin lacks it, the panel reports why instead of showing an empty list.

Protected permission sets. Permission sets installed by other managed packages may not be readable, and their permissions still apply. The header reports how many were skipped, so the admin knows the grid is a lower bound rather than a complete answer.

Sharing rules and record-level access. Out of scope. __Share queries can be enormous and reading org-wide defaults properly needs a callout. We show the org-wide default only.

Handling scale and governor limits

Keyset pagination rather than OFFSET. SOQL OFFSET is capped at 2,000 rows, which silently truncates user lists in large orgs. PEKeysetCursor encodes an opaque cursor carrying the sort value and the last row Id. The Id tiebreaker is mandatory: the sort value is never unique, and without it rows duplicate or vanish between pages. Two tests cover this — PEUserSearchServiceTest.pagingCollectsEveryRowExactlyOnce walks every page and asserts each row appears exactly once, and lastLoginSortIsAcceptedAndPages covers the date-sorted branch. Cursors are integrity-checked; PermissionExplorerTest.tamperedCursorIsRejected asserts a modified cursor is refused.

Query chunking. A user can have hundreds of effective permission-set parents, more than a single SOQL bind list should carry. PESoqlChunker splits them into batches, and assertQueryBudget checks the resulting query count against the governor limit before any

query runs, throwing a typed retryable error if it would not fit. This is why several methods query inside a loop, and it is the subject of five of our six analyser suppressions — see the false positives document.

One composite endpoint. getObjectDetail returns object permissions, field permissions, record types and sharing model in a single round trip, so switching sections in the interface is pure client-side rendering.

Per-section error boundaries. A failure loading fields populates a typed error on the field section only. The object permissions panel that already rendered correctly is not blanked.

Using the tool

Install and set up

1. Install the package. Admins Only is the recommended setting.
2. Setup → Permission Sets → PermissionCheckerPermissionSet → Manage Assignments, and assign it to the administrators who should be able to inspect other users.
3. Look for **Who Sees What** in the App Launcher. If it is not there: Setup → Profiles → your profile → Assigned Apps → Edit → enable Who Sees What → Save. App assignment lives in profile metadata and cannot be granted by a permission set.
4. Open App Launcher → Who Sees What → Permission Explorer.

Before any user is selected the tool shows an onboarding panel repeating these steps, listing what it can tell you, and stating that use is logged.

Inspect a user

1. In the user column, search by name, username, alias or email, or filter by profile, role, or user type. Sort by name or last login. Results are keyset paginated.
2. Select a user. The header shows profile, role, licence, last sign-in, and counts of assigned permission sets and groups. Each count opens a drill-down with links to the matching Setup page.
3. The app column lists apps visible to that user, each marked “visibility estimated.”
4. Open an app to see the objects and tabs reachable from it, with create, read, update, delete, view all and modify all chips and the granting source for each. Objects the user can reach by data access but which have no tab in that app are listed separately — that gap is a common source of “why can they see this?” questions, so we surface it rather than omitting it.
5. Expand an object for three tabs. Fields shows readable and editable per field with granting sources, and an explicit reason where field-level security does not apply at all, such as formula or required fields. Record types shows the org-level list under the not-analysed notice. Origin shows where the object access came from.
6. Copy link shares the current selection. Refresh data bumps a cache version token rather than clearing the partition, so one admin refreshing does not cost every other admin their warm cache.

Checking the audit log

```
SELECT twinforce__Inspector_User__c, twinforce__Subject_User__c,
       twinforce__Action__c, twinforce__Inspected_At__c
FROM twinforce__PE_Inspection_Log__c
ORDER BY CreatedDate DESC
```

Test coverage

74 Apex tests ship with the package. All pass. Coverage is 88%, and Salesforce reports the coverage requirement as met.

Class	Coverage
PEAccessGate	100%
PEConstants	100%
PEDto	100%
PermissionExplorerException	100%
PEUserSearchService	100%
PEKeysetCursor	97%
PEInspectionAuditLogger	94%
PermissionExplorerController	93%
PEAccessQueryService	89%
PEMetadataCatalog	88%
PEFlsClassifier	86%
PEPermissionResolver	84%
PESoqlChunker	81%
PECacheManager	80%
PEUserDetailService	57%

The tests assert security properties, not only that the happy path works:

- unauthorisedCallerIsRejectedByEveryEntryPoint — the gate holds on every entry point
- unauthorisedUserIsRejectedBeforeAnyQuery — rejection precedes any SOQL
- quoteInTermIsBoundNotInterpolated — a quote in a search term is bound, not interpolated
- disallowedSortFieldIsRejected — the sort allowlist holds
- tamperedCursorIsRejected — cursor integrity
- pagingCollectsEveryRowExactlyOnce — no duplicated or skipped rows across pages
- mutingSubtractsAGrantedPermission, mutingDoesNotInventAGrant, mutingDoesNotLeakAcrossPermissions — muting correctness
- absentRowMeansNoAccessNotUnknown — absence means no access, never unknown

Why PEUserDetailService sits at 57%. Two regions of it cannot be covered by any test, and we documented this in the class header rather than leaving it looking like neglect. The not-readable branch depends on Schema.isAccessible() returning false, which cannot be arranged

from a test because System.runAs enforces record sharing but not object permissions. The query loop needs LoginHistory rows, which cannot be inserted in a test context. That is a floor, not a gap we have yet to close.

Static analysis

Salesforce Code Analyzer plugin 5.15.0, using the AppExchange and Security rulesets: **0 violations.**

```
sf code-analyzer run --workspace force-app --config-file code-
analyzer.yml \
  --rule-selector AppExchange --rule-selector Recommended:Security
→ Found 0 violations.
```

Six @SuppressWarnings annotations remain in PEAccessQueryService. None of the rules they suppress appear in the AppExchange or Security rulesets — we verified that by removing all six and re-running the gate scan, which still returned zero. Each is explained in the false positives document and in the code at the point of suppression.